

ToSh Cheat Sheet: Units, Dates, And Time

StorageSize, TimeSpan, TemporalAmount, date math, and physical quantities

spec + runtime behavior

Temporal Literals

5s
100ms
2m30s
1w2d4h5m23s
1y2mo
2026-03-25

Literal	Runtime type
2d, 5s	TimeSpan
1y2mo	TemporalAmount
2026-03-25	DateTimeOffset

```
date now
date parse 2026-03-23T14:05:00Z
date date-only 2026-03-23T14:05:00Z
date time-only 2026-03-23T14:05:00Z
timespan 1w2d
```

Date Arithmetic

```
var now = (date now)
var yesterday = (($now) - 1d)
var nextWeek = (($now) + 1w)

var jan31 = (date 2026-01-31)
var febEnd = (($jan31) + 1mo)
echo $febEnd.Month $febEnd.Day
```

Calendar units like mo and y produce TemporalAmount; pure clock units produce TimeSpan.

Storage Sizes

```
1kb
10mb
2.5gb

ls -la | sort Size
ls -la | where _.Size >= 1kb | get Name
ls -la | where _.Size? > 1000 | get Name
```

- StorageSize values compare naturally in pipelines.
- They can be compared against either numbers or size literals.
- Display is configurable from \$tosh.Config.Display.

Quantity Literals

100‘m
5‘kg
9.8‘m/s^2
2.5‘kWh
72‘degF
90‘deg

Piece	Meaning
100‘m	number + backtick + unit expression
m/s^2	division and exponent are allowed
km, MB	SI prefixes resolve automatically

```
100‘m + 50‘m
100‘km + 500‘m
100‘m / 20‘s
5‘kg * 9.8‘m/s^2
```

Quantities enforce dimensional analysis: compatible dimensions add/compare cleanly; incompatible ones raise an error.

Named Quantity Categories

Category	Example units
Length	m, km, ft, mi
Mass	kg, g, lb
Duration	s, ms, min, hr
Temperature	K, degC, degF
DataSize	B, kB, MB, GB
Speed	m/s, km/h, mph
Energy	J, kJ, kWh, BTU
Angle	rad, deg

Quantity Members

```
var d = 5‘km
echo $d.Magnitude
echo $d.UnitSymbol
echo $d.BaseValue
echo $d.CategoryName
```

- Magnitude: numeric value in the original unit
- UnitSymbol: unit text as entered
- BaseValue: SI-normalized value
- CategoryName: named physical category when recognized

Temporal Pipelines

```
func recentFiles(age: TimeSpan) {
    ls -la | where _.Type == file | where _.Modified > ((date now) - $age)
}

recentFiles 7d | get { Name, Modified, Size }
```

```
date parse 2026-03-23T14:05:00Z -d -t
date parse 2026-03-23T14:05:00Z | cast dateonly
date parse 2026-03-23T14:05:00Z | cast timeonly
```

Date Helpers

```
date today
date tomorrow
date utc-now
date from-unix 1742738700
```

Other Intrinsic Literals

```
127.0.0.1
::1
```

- Bare IPv4 and IPv6 values parse as System.Net.IPAddress.
- Typed parameters can coerce string input into ipaddress values.

Practical Examples

```
# files newer than two days ago
ls -la | where _.Modified > ((date now) - 2d)

# average speed for a run
echo (5‘km / 24‘min)

# filter by size
find . -name *.log | where _.Size >= 10mb
```

Display Tuning

```
view datetime table unix
view duration table seconds
view size bytes
view dateonly scalar relative
```

Bridge behavior: ToSh can promote plain TimeSpan and StorageSize values when they interact with the unit system, so date math and size comparisons stay ergonomic in real shell pipelines.

Bridge Types

Form	Meaning
5s, 2h30m	TimeSpan clock durations
1y2mo	TemporalAmount calendar duration
10mb	StorageSize byte-oriented size
5'km	Quantity with dimensional analysis
127.0.0.1	System.Net.IPAddress literal

- Use storage literals for files and memory, unit literals for physics and engineering math.
- `date add/sub` accepts a `TemporalAmount`, so both `2d` and `1mo` work.
- `sum` and `avg` handle numeric, `StorageSize`, and `TimeSpan` streams directly.

Derived Quantity Families

Category	Examples
Area / Volume	m^2, acre, L, gal
Acceleration / Force	m/s^2, N, lbf
Power / Pressure	W, kW, Pa, psi
Charge / Resistance	C, Ah, V, ohm
Density / FlowRate	kg/m^3, L/min, gal/min
Angle / AngularVelocity	deg, rad, deg/s

Dimension Math

```
var force = (12'kg * 9.81'm/s^2)
var work = ($force * 3'm)
var power = ($work / 10's)
```

```
echo $force
echo $work
echo $power
```

```
echo (60'mph > 25'm/s)
echo (7850'kg/m^3 * 0.01'm^3)
echo (2'L / 30's)
```

Calendar-Aware Recipes

```
date today
date tomorrow
date yesterday
date utc-now
date from-unix-ms 1742738700000
```

```
date today | date add 1mo
date 2026-03-25 | date sub 2w
date parse 2026-03-23T14:05:00Z | date time-only
```

- Month/year amounts clamp naturally at month ends.
- Pipe a date into `date add` / `date sub` when you want calendar logic without extra variables.
- Use `-d` and `-t` when you want both projections emitted at once.

Temporal Filtering

```
ls -la |
where _.Type == file |
where _.Modified > ((date now) - 12h) |
sort Modified | reverse
```

```
find . -name *.log |
where _.Modified < ((date now) - 30d) |
get { Name, Modified, Size }
```

Size And Duration Aggregates

```
echo 1kb 2kb 3kb | sum
echo 1s 2s 3s | avg
ls -la | where _.Type == file | summarize --sum Size
```

Aggregation is strongest on shell-native temporal and size types. Quantities are best used for direct formula-style calculations and comparisons.

Display Profiles

```
view dateonly long
view timeonly 24h
view duration long
view size auto
```

Scientific And Ops Examples

```
# network throughput from bytes / time
echo (120'MB / 30's)
```

```
# battery energy over time -> power
echo (2.5'kWh / 30'min)
```

```
# rotation rate
echo (180'deg / 2's)
```

```
# compare mixed units
echo (5'mi > 8'km)
```

```
# temperature math stays typed
echo 72'degF
echo 22'degC
```

Working With Dates As Types

```
var dt = (date now)
var d = ($dt | cast dateonly)
var t = ($dt | cast timeonly)
```

```
echo $d
echo $t
```

```
type-of (date now)
type-of 2d
type-of 1y2mo
type-of 5'km
```

Practical File Retention

```
func staleFiles(age: TimeSpan, min: StorageSize) {
    ls -la |
    where _.Type == file |
    where _.Modified < ((date now) - $age) |
    where _.Size >= $min
}
```

```
staleFiles 14d 100mb | get { Name, Size, Modified }
```

Mental Model

- `2d` is shell time, `2'm` is physical length, and `2mb` is storage.
- Calendar durations (`mo`, `y`) are not just scaled seconds; they preserve calendar semantics.
- Quantities keep dimensions honest, which makes shell calculations safer than raw doubles.